Laboratório Funcional · Aula 12

Gabriel Cesar Gravena · RA 24159150-2 · Turma A
Gerado em 16/10/2026, 20:23:41 · tempo registrado no cronômetro: 55:49

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

def agora(): return datetime.now() : Ela depende de um fator externo mutável no tempo, uma função pura deve ser determinística, ou seja, para os mesmos parâmetros de entrada, deve retornar exatamente o mesmo valor em qualquer momento, como datetime.now() muda a cada segundo, a função quebra essa regra
def registra(msg): print(msg): Ela produz um efeito colateral, ou seja, funções puras apenas computam e retornam valores sem interagir com o mundo exterior, o comando print(msg) altera o estado da tela do computador, o que a torna impura

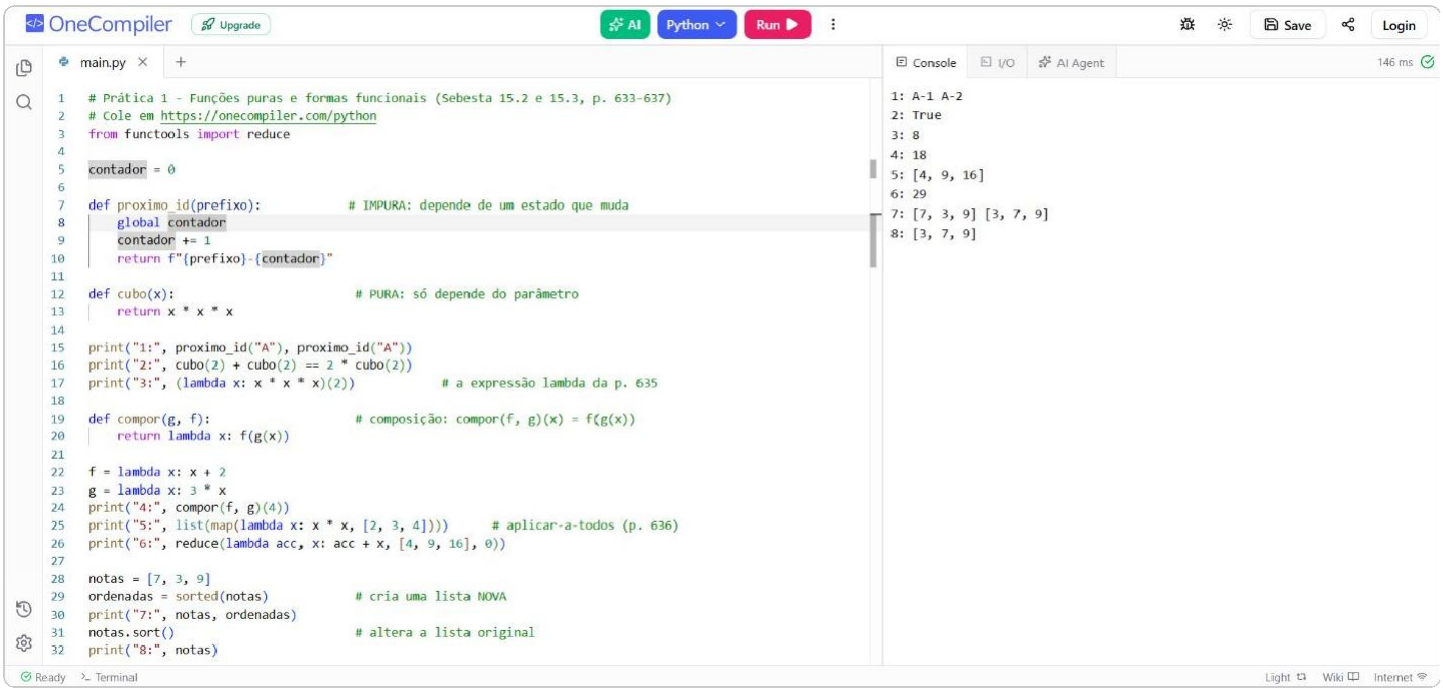
Prática 1: Funções puras e formas funcionais

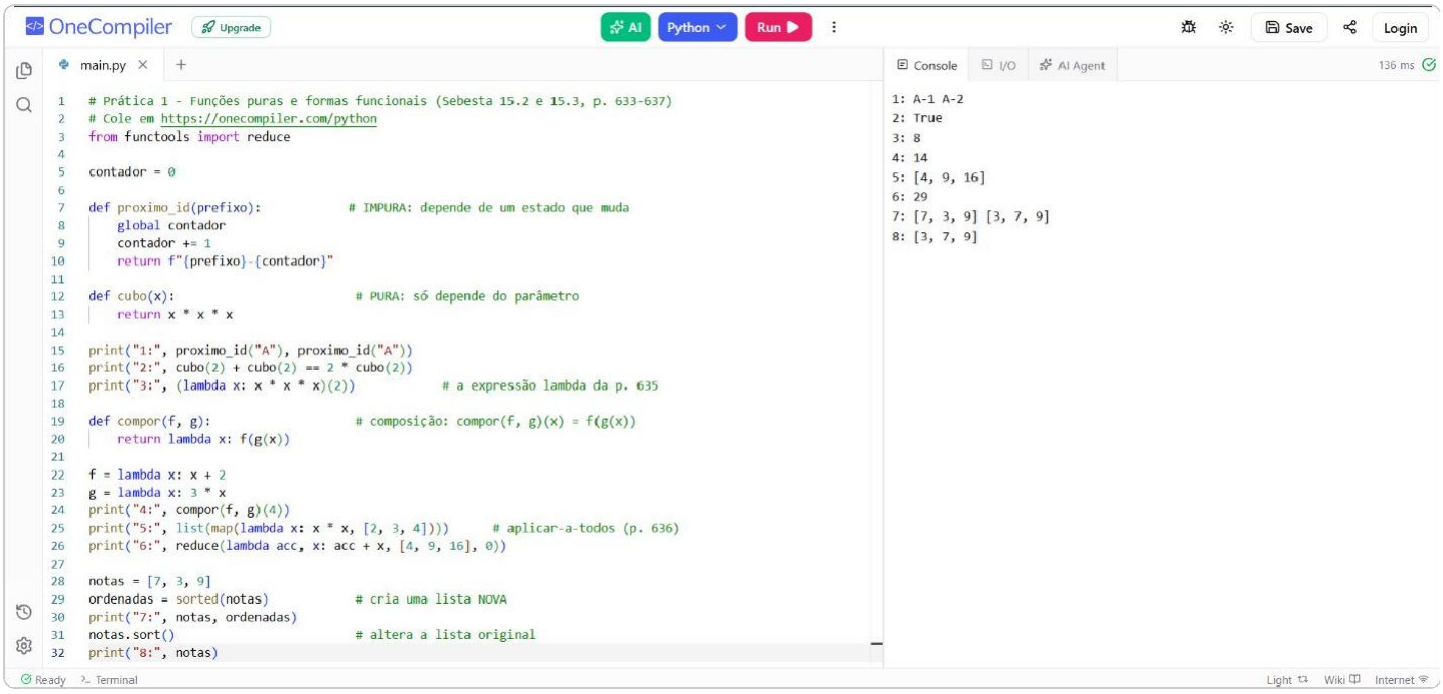
Previsão: O que as linhas 1, 4 e 7 vão imprimir?

Na linha 1, o programa imprime 1: A-1 A-2 porque a função proximo_id incrementa o contador global a cada chamada, na linha 4, imprime 4: 14 ao calcular a composição f(g(4)), onde g(4) = 12 e f(12) = 14, na linha 7, imprime 7: [7, 3, 9] [3, 7, 9] porque sorted() gera uma nova lista ordenada sem alterar a lista original notas

Saídas (original e com compor(g, f))

não respondido





1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

A função proximo_id("A") devolve valores diferentes porque depende e altera uma variável global mutável, tornando o seu resultado dependente do histórico de chamadas anteriores, isso dificulta os testes porque as chamadas deixam de ser independentes, exigindo a reinicialização do estado global para garantir resultados previsíveis

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

Calculando compor(f, g)(4), temos f(g(4)) = f(3 x4) = f(12) = 12 + 2 = 14, já em compor(g, f)(4), temos g(f(4)) = g(4 + 2) = g(6) = 3 x6 = 18, e a diferença ocorre porque a composição de funções não é comutativa, alterando a ordem em que as transformações são aplicadas sobre o valor

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

A função sorted(notas) retorna uma nova lista ordenada mantendo a lista original intocada, enquanto notas.sort() modifica a própria lista original no local e retorna None, o sorted(notas) segue o estilo funcional porque respeita a imutabilidade dos dados e evita efeitos colaterais

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

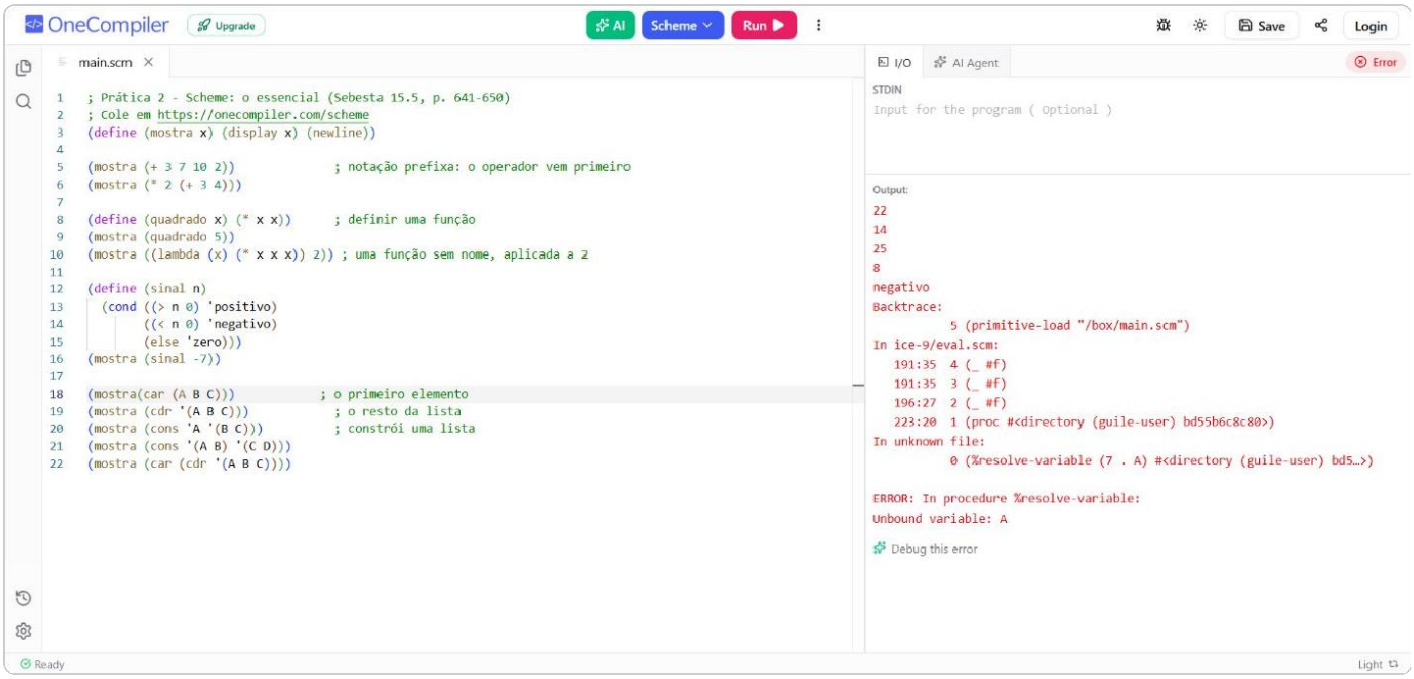
(B, C)

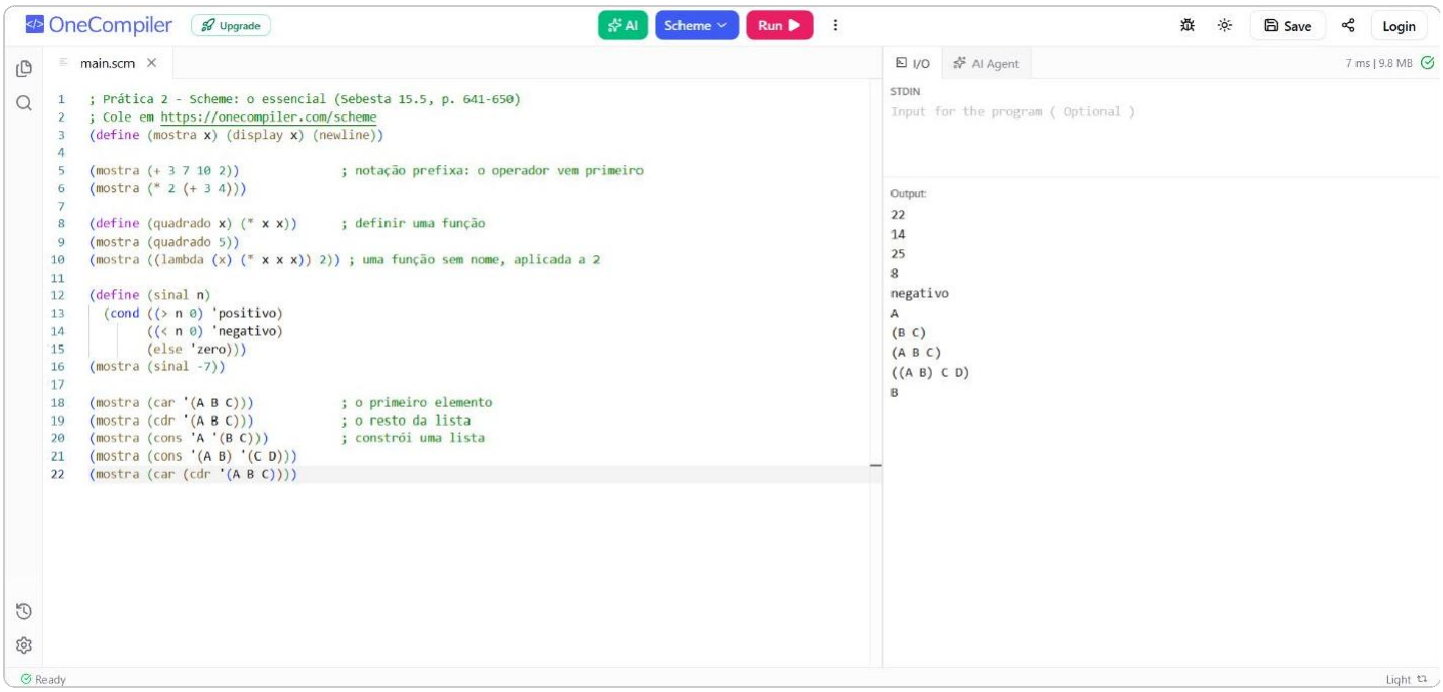
Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

não respondido





2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo serve para indicar que a lista é um dado literal e não deve ser avaliada como uma expressão executável, sem ele, o Scheme tentou interpretar A como o nome de uma função a ser chamada, passando B e C como argumentos.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C)))) resultado C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 3: comum em 6 passos, até 2 multiplicações esperando (expressão de até 24 caracteres); em cauda em 5 passos, nada esperando (até 16 caracteres).
n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).
n = 5: comum em 10 passos, até 4 multiplicações esperando (expressão de até 36 caracteres); em cauda em 7 passos, nada esperando (até 18 caracteres).
n = 6: comum em 12 passos, até 5 multiplicações esperando (expressão de até 42 caracteres); em cauda em 8 passos, nada esperando (até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

Em Scheme vai funcionar, porque a função é recursiva em cauda e o Scheme reaproveita o registro de ativação, então imprime 1000000, em Python não vai, porque ele não otimiza a chamada em cauda e cada chamada empilha um registro novo, dando RecursionError por volta de mil chamadas

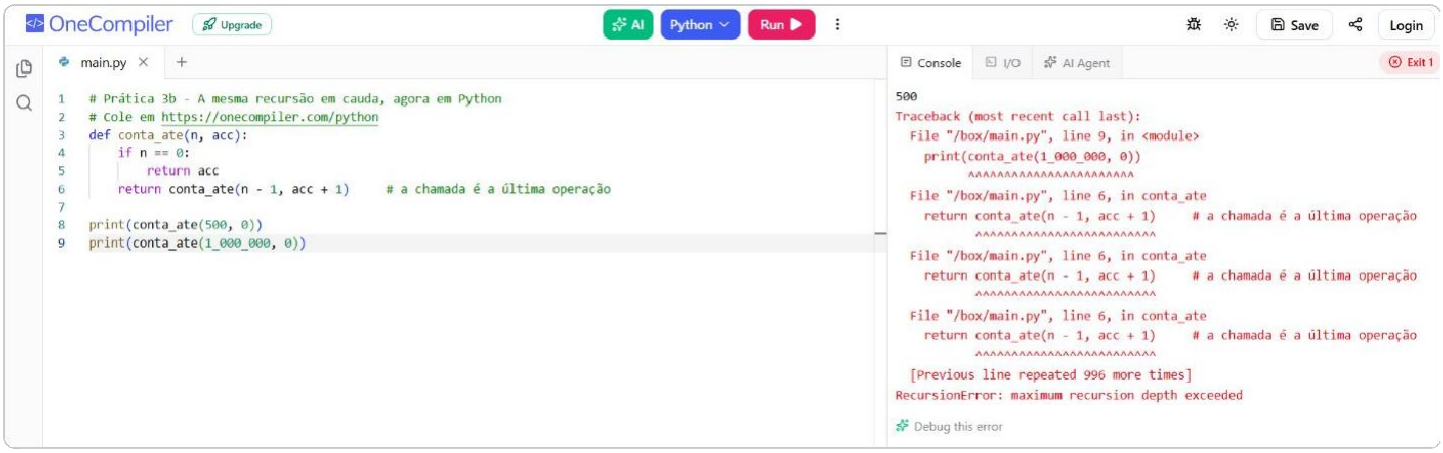
Saída do Scheme

não respondido



Saída do Python (as últimas linhas bastam)

não respondido



3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Cresce a expressão e o número de multiplicações pendentes, porque na recursão comum a multiplicação fica esperando a volta de cada chamada, na recursão em cauda não cresce, porque o resultado parcial vai no parâmetro acumulador e nada fica esperando

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme otimiza a chamada em cauda, ou seja, como nada fica esperando, ele reaproveita o registro de ativação e usa espaço constante, ja o Python não faz essa otimização, então cada chamada empilha um registro novo e, ao passar do limite de recursão, dá RecursionError

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho:  3   ok
soma-quadrados:   14   ok
pares:  (2 4 6)   ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (cond ((null? lis) 0)
        (else (+ 1 (tamanho (cdr lis))))))

(define (soma-quadrados lis)
  (cond ((null? lis) 0)
        (else (+ (* (car lis) (car lis)) (soma-quadrados (cdr lis))))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

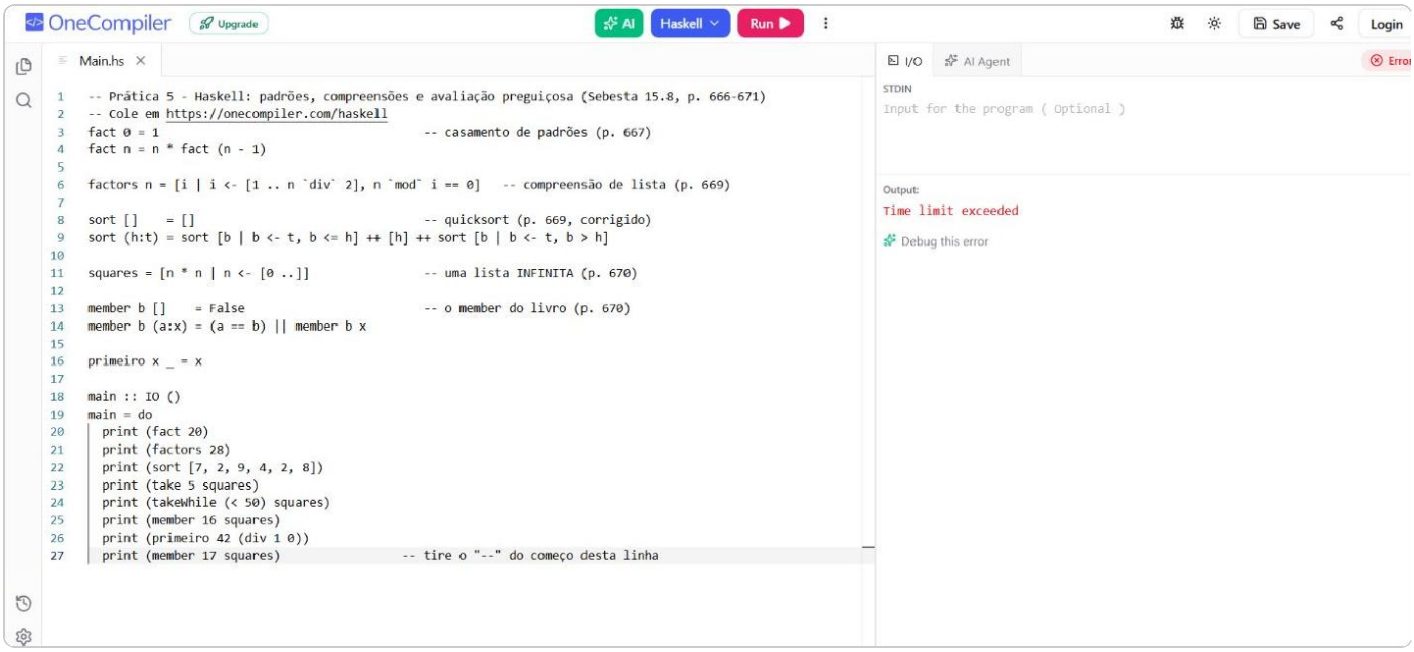
A parte mais difícil foi o pares, porque tem um caso a mais, entao é só coloca o elemento com cons se for par, senão ignora e segue, para o caso da lista vazia, car para olhar o primeiro elemento e cdr para chamar a função de novo com o resto da lista, no tamanho o car não é necessário, só conto 1 e sigo com o cdr

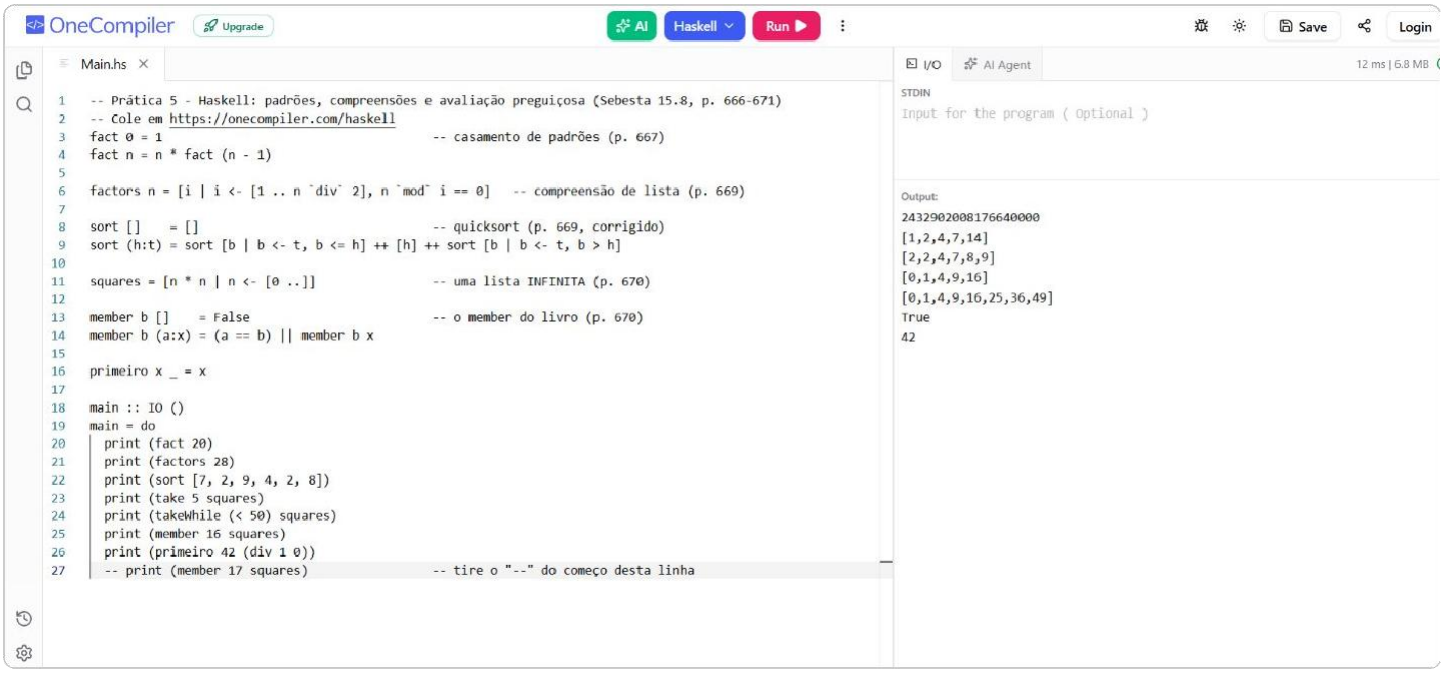
Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?
take 5 squares imprime [0,1,4,9,16], porque o Haskell tem avaliação preguiçosa e só calcula os 5 primeiros elementos da lista infinita, primeiro 42 (div 1 0) imprime 42, porque a função não usa o segundo parâmetro, então o div 1 nunca é calculado e a divisão por zero não acontece

Saídas (original e com member 17)

não respondido





5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell usa avaliação preguiçosa, só calcula um elemento da lista quando ele é necessário, então take 5 squares gera apenas os 5 primeiros e ignora o resto da lista infinita, o primeiro 42 (div 1 0) não deu erro porque a função não usa o segundo parâmetro, então o div 1 0 nunca é calculado

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

O member 17 squares ficou rodando sem parar, porque 17 não é quadrado perfeito e a busca continua comparando com cada elemento da lista infinita sem nunca achar, o member 16 squares terminou porque o 16 aparece e a busca para ali, entao para corrigir, como a lista está em ordem crescente, dá para desistir quando o elemento passa do valor procurado, por exemplo com member 17 (takeWhile (<= 17) squares), ou definindo um member que retorna False quando a > b

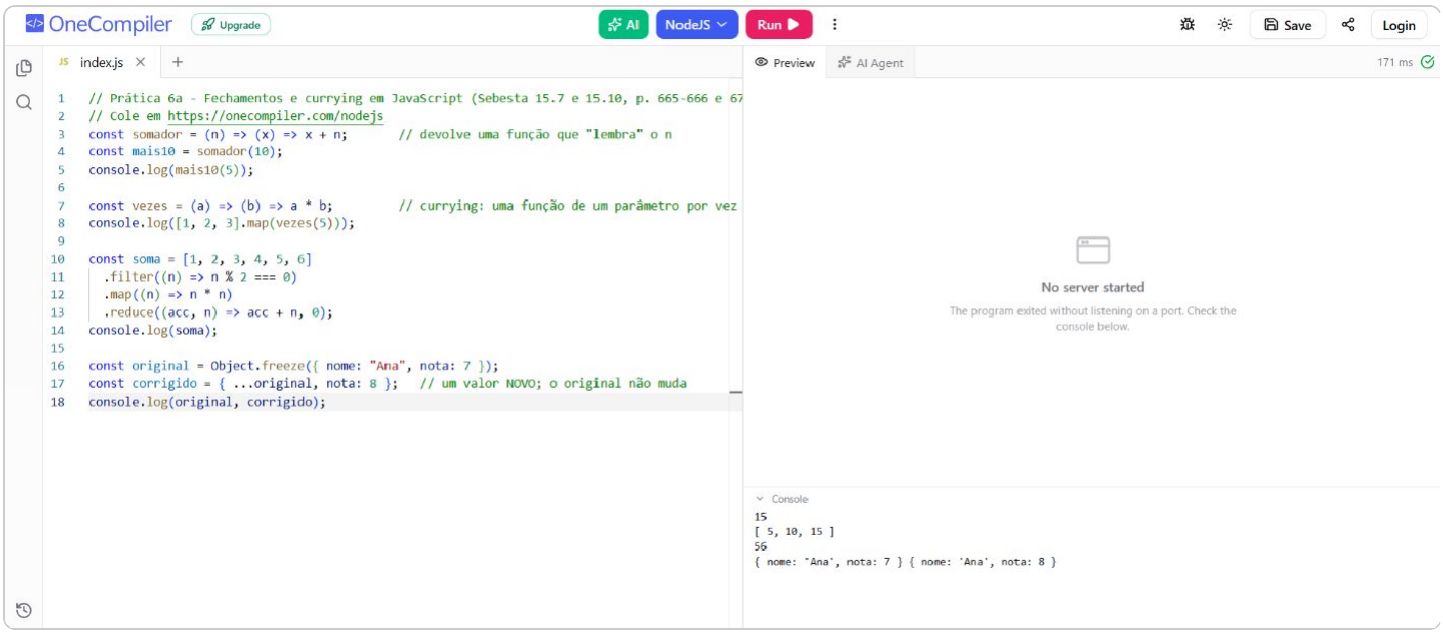
Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) imprime 15, porque mais10 é um fechamento que lembra o n = 10 e soma com o 5. [1, 2, 3].map(vezes(5)) imprime [5, 10, 15], porque vezes(5) devolve uma função que multiplica por 5, e o map aplica essa função a cada elemento

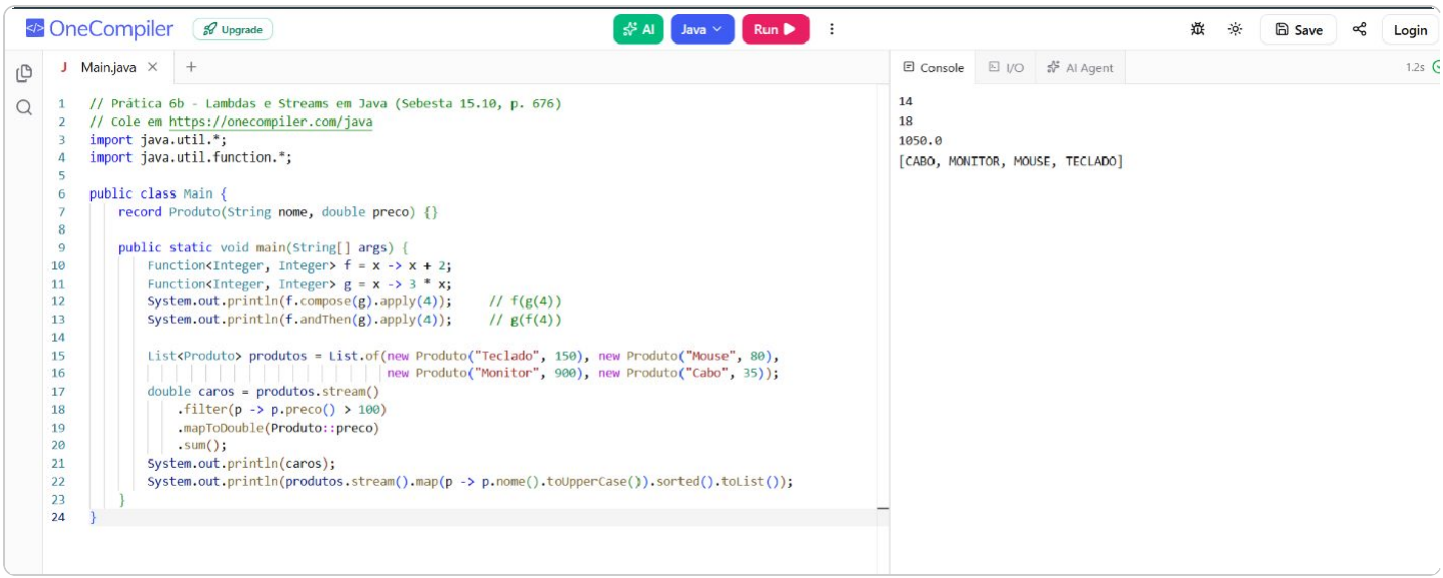
Saída do JavaScript

não respondido



Saída do Java

não respondido



6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

Porque a função devolvida é um fechamento, ela guarda uma ligação com o ambiente onde foi criada, e a variável capturada n = 10 continua existindo enquanto essa função existir, mesmo depois de somador(10) terminar

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

Porque a ordem de aplicação é diferente. f.compose(g).apply(4) calcula f(g(4)) = f(12) = 14, e f.andThen(g).apply(4) calcula g(f(4)) = g(6) = 18, o compose é o que corresponde ao compor(f, g) da Prática 1, já que aplica o g primeiro e depois o f, como na composição matemática f(g(x))

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
const aprovados = alunos
  .filter((a) => a.nota >= 6)
  .map((a) => a.nome.toUpperCase());
console.log(aprovados.join(", "));
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
com_desconto = [p * 0.9 if p > 100 else p for p in precos]
print(com_desconto)
```

Saídas das suas versões

```
ANA, CARLA
[80.0, 135.0, 90.0, 180.0]
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão funcional é mais fácil de ler, porque diz direto o que se quer sem variável acumuladora nem controle de índice, e o código fica mais curto, a versão com laço é mais fácil de depurar, porque dá para colocar um print ou um breakpoint dentro do laço e ver o valor intermediário a cada passo, enquanto no pipeline é preciso quebrar a cadeia ou inserir algo extra para ver o que sai de cada etapa

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em Haskell é mais fácil de entender, porque diz o que se quer em uma linha, sem índice nem acumulador. Aceitaria o custo de 2 vezes mais tempo em sistemas de regras de negócio ou relatórios, onde clareza e menos erros importam mais

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi ver que o Scheme consegue fazer um milhão de chamadas recursivas sem problema, enquanto o Python, com a mesma função, para com RecursionError por volta de mil

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro de impressão está na primeira compreensão [b | b <- t, b <- h], o segundo b <- h é outro gerador, não um filtro, e ele diz que b é tirado de h, tratando h como se fosse uma lista, como h é um elemento, o compilador infere que h deveria ser uma lista e dá erro de tipo, reclamando que não consegue casar o tipo [a] com o tipo do número. A correção é trocar por um filtro: [b | b <- t, b <= h]